

Simulation-based Inference with the Python Package sbijax

Simon Dirmeier^{1,2}, Antonietta Mira^{3,4}, and Carlo Albert⁵

¹ Swiss Data Science Center, Zurich, Switzerland ² ETH Zurich, Zurich, Switzerland ³ Università della Svizzera italiana, Switzerland ⁴ University of Insubria, Italy ⁵ Swiss Federal Institute of Aquatic Science and Technology, Switzerland

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Richard Littauer](#) 

Reviewers:

- [@aneeshnaik](#)
- [@dosvk](#)

Submitted: 19 March 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

In natural sciences like astrophysics, biology or neuroscience, models can be frequently formulated as simulator functions, i.e., computer programs that stochastically map a parameter vector θ to synthetic data x . Even though simulators of this kind can represent realistic descriptions of real-world data-generating processes, they often do emit intractable probability density functions. However, statistical inference of the parameters of a simulator requires the likelihood $p(x|\theta)$ to be tractable, such that inference algorithms like Markov chain Monte Carlo can be applied. Simulation-based inference (SBI) circumvents this by only requiring the ability to sample from the simulator: parameters θ are drawn from the prior distribution $p(\theta)$, an observable x is drawn from the simulator $p(x|\theta)$, and the resulting pairs are used to approximate the posterior distribution. In computational statistics, neural SBI describes an emerging family of methods for Bayesian inference for simulator models that use neural networks as surrogate models. Here we introduce sbijax, a Python package that implements a wide variety of state-of-the-art methods in neural SBI using a user-friendly programming interface. Targeted at domain scientists, e.g., in computational physics or computational biology, and SBI researchers, sbijax offers high-level functionality to quickly construct SBI estimators and compute posterior distributions with only a few lines of code. In addition, the package provides functionality for conventional approximate Bayesian computation, to compute model diagnostics, and to automatically estimate summary statistics. By virtue of being entirely written in JAX, sbijax is extremely computationally efficient, allowing rapid training of neural networks and executing code automatically in parallel on both CPU and GPU.

Statement of Need

Modern approaches to neural SBI utilize recent developments in neural density estimation or score-based generative modelling to build surrogate models to approximate Bayesian posterior distributions. Similarly to conventional methods, such as approximate Bayesian computation (ABC, Sisson et al. (2018)) and its sequential (SMC-ABC; Sisson et al. (2007), Beaumont et al. (2009)) and annealing-based (e.g., SABC, Albert et al. (2015)) variants, neural SBI methods infer this posterior distribution by first simulating synthetic data and then numerically constructing an appropriate approximation to this pseudo data set. SBI methods are attractive for a couple of reasons. On the one hand, this family of methods has been shown to be more computationally efficient and often more accurate than ABC methods, in particular for smaller simulation budgets. On the other hand, SBI allows to easily amortize inference, i.e., to infer the posterior distribution for multiple different observations once a neural model has been trained.

Here we propose sbijax, a Python package implementing state-of-the-art methodology of

neural simulation-based inference. While the main focus of the package is the implementation of recent algorithms to make them available to practitioners, e.g., Albert et al. (2025) or Gloeckler et al. (2024), sbijax also implements common methods from approximate Bayesian computation, e.g., SMC-ABC, to have the entire SBI toolbox in one efficient package (see Table 1 for an overview). In addition, sbijax provides functionality for model diagnostics and Markov Chain Monte Carlo (MCMC) sampling.

The package uses the high-performance computing framework JAX as a backend (Bradbury et al., 2018). Using JAX has several advantages, including a) that it uses the same syntax as numpy (Harris et al., 2020) which enables a seamless transition for applied scientists who already are familiar with it, and b) that empirical evaluations have shown that JAX can be significantly faster than PyTorch (see, e.g., Phan et al. (2019)). Our package heavily builds on libraries from the JAX-verse and common Bayesian inference tools. Specifically, we use Haiku (Hennigan et al., 2020) to construct and train neural networks, surjectors for normalizing flow based density estimation (Dirmeier, 2024), TensorFlow Probability (Dillon et al., 2017) to define statistical distributions, and BlackJAX (Cabezas et al., 2024) for posterior sampling using Markov Chain Monte Carlo.

Table 1: Implemented SBI methods in sbijax .

Model	Class name	Reference
Sequential Monte Carlo ABC	SMCABC	Beaumont et al. (2009)
Simulated annealing ABC	SABC	Albert et al. (2025)
Neural likelihood estimation	NLE	Papamakarios et al. (2019)
Surjective neural likelihood estimation	SNLE	Dirmeier et al. (2025)
Automatic posterior transformation	NPE	Greenberg et al. (2019)
Contrastive neural ratio estimation	NRE	Miller et al. (2022)
Flow matching posterior estimation	FMPE	Wildberger et al. (2023)
Posterior Score Estimation	NPSE	Sharrock et al. (2024)
All-In-One Posterior Estimation	AIO	Gloeckler et al. (2024)
Consistency model posterior estimation	CMPE	Schmitt et al. (2023)
Neural approximate sufficient statistics	NASS	Chen et al. (2021)
Neural approximate slice sufficient statistics	NASSS	Chen et al. (2023)

State of the field

While a plethora of different models has been proposed in the recent literature, the development of adequate software packages has not followed at the same pace, and only few packages exist that allow modelers to use these methods. Most prominently, the Python package sbi (Tejero-Cantero et al., 2020) implements several approaches for neural simulation-based inference, such as a neural posterior, likelihood-ratio, and likelihood estimation (Cranmer et al., 2020) utilizing a PyTorch backend (Paszke et al., 2019). The package additionally provides an API for model diagnostics, such as posterior predictive checks, effective sample size computations and simulation-based calibration. However, the package lacks implementations of recent developments which pose the state-of-the-art in the field, such as by Chen et al. (2023), Dirmeier et al. (2025) or Albert et al. (2025). BayesFlow (Radev et al., 2023) offers a wide variety of neural methods, but is similarly limited in the set of estimators it is implementing. For approximate Bayesian computation, several Python packages are available. In particular abcpy (Dutta et al., 2021) implements a multitude of different ABC algorithms. However, none of these packages implement modern (neural) SBI methods.

Software design

sbijax is designed as a modular and extensible toolbox for SBI in JAX, combining a functional programming philosophy with a flexible interface for both expert users and domain scientists. Its design is guided by the following principles:

- 1) Faithful alignment with JAX's functional paradigm. Sbijax implements a fully functional API in the idiom of Haiku: every method is a factory function that returns a tuple of pure functions. All non-permanent state variables, i.e., neural network parameters and optimizer states, are passed explicitly instead of being held by an object. A neural estimator, e.g., `nle(make_maf(.))`, returns training primitives and a sampling function which are consumed by the generic drivers `train` and `sample`, while an ABC method, e.g., `sabc(prior, simulator)`, returns a sampler that draws from the approximate posterior directly. No function mutates internal state facilitating composability, and ensures compatibility with JAX transformations such as `jit`, `vmap`, and `pmap`.
- 2) Separation of inference methods and neural network implementations. sbijax focuses on implementing SBI algorithms, while neural network components are defined using Haiku. This allows users to construct models directly with Haiku, and to incorporate probabilistic building blocks from Distrax (DeepMind et al., 2020) and surjectors (Dirmeier, 2024). As a result, model definitions remain flexible and low-level, while seamlessly interoperating with the broader JAX ecosystem.
- 3) A deliberately low-level interface. An inference workflow typically written as a short sequence of ordinary function calls, where the inputs and outputs of every step, e.g., the simulated data set and the network parameters, are managed by the developer explicitly. This asks developers to be deliberate about the workflow they are executing rather than calling the member functions of an object whose internal state is (often) opaque to them. We consider this trade-off worthwhile for a scientific software tool, since it makes a method transparent to the practitioner using it (albeit it induces some additional complexity) and straightforward to modify for a researcher extending it.
- 4) Support for extensibility and research. Sbijax is structured to facilitate experimentation with new SBI methods. Its modular design allows components such as neural architectures, training objectives, and sampling strategies to be easily replaced or extended, making it suitable as both a research framework and a practical toolbox. Since every method reduces to the same small set of primitives, namely an initialization, a gradient step, an evaluation step and a sampling function, a newly implemented objective can be trained and sampled from with the existing drivers without any further changes.
- 5) Accessibility for domain scientists. In addition to its flexibility, sbijax includes pre-implemented models with sensible defaults, enabling use without in-depth expertise in deep learning. In the simplest case, users only need to define a prior and a simulator to run inference workflows, often in as few as five lines of code.

Research impact statement

Sbijax has already been used extensively in the Machine Learning literature. Dirmeier et al. (2025) and Dirmeier & Mira (2025) proposed novel SBI methods for surjective neural likelihood estimation and posterior estimation using causal constraints, respectively, where they used sbijax heavily for their experimental evaluations. Ulzega et al. (2025) used sbijax to infer the posterior distribution of a complicated Bayesian model from the astrophysics literature. Albert et al. (2025) developed a novel ABC method that uses sbijax for model evaluation.

119 **AI usage disclosure**

120 No GenAI or other AI tools have been used in writing the software or this manuscript.

121 **Acknowledgements**

122 This research was supported by the Swiss National Science Foundation (Grant
123 No. 200021_208249).

124 **References**

- 125 Albert, C., Künsch, H. R., & Scheidegger, A. (2015). A simulated annealing approach to
126 approximate Bayes computations. *Statistics and Computing*, 25, 1217–1232.
- 127 Albert, C., Ulzega, S., Dirmeier, S., Scheidegger, A., Bassi, A., & Mira, A. (2025). Simulated
128 annealing ABC with multiple summary statistics. *arXiv Preprint arXiv:2505.23261*.
- 129 Beaumont, M. A., Cornuet, J.-M., Marin, J.-M., & Robert, C. P. (2009). Adaptive approximate
130 Bayesian computation. *Biometrika*, 96(4), 983–990.
- 131 Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G.,
132 Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable*
133 *transformations of Python+NumPy programs* (Version 0.3.13). [http://github.com/google/](http://github.com/google/jax)
134 [jax](http://github.com/google/jax)
- 135 Cabezas, A., Corenflos, A., Lao, J., & Louf, R. (2024). BlackJAX: Composable Bayesian
136 inference in JAX. *arXiv Preprint arXiv:2402.10797*.
- 137 Chen, Y., Gutmann, M. U., & Weller, A. (2023). Is learning summary statistics necessary for
138 likelihood-free inference? *Proceedings of the 40th International Conference on Machine*
139 *Learning*.
- 140 Chen, Y., Zhang, D., Gutmann, M. U., Courville, A., & Zhu, Z. (2021). Neural
141 approximate sufficient statistics for implicit models. *International Conference on Learning*
142 *Representations*.
- 143 Cranmer, K., Brehmer, J., & Louppe, G. (2020). The frontier of simulation-based inference.
144 *Proceedings of the National Academy of Sciences*, 117(48), 30055–30062.
- 145 DeepMind, Babuschkin, I., Baumli, K., Bell, A., Bhupatiraju, S., Bruce, J., Buchlovsky, P.,
146 Budden, D., Cai, T., Clark, A., Danihelka, I., Dedieu, A., Fantacci, C., Godwin, J., Jones,
147 C., Hemsley, R., Hennigan, T., Hessel, M., Hou, S., ... Viola, F. (2020). *The DeepMind*
148 *JAX Ecosystem*. <http://github.com/deepmind>
- 149 Dillon, J. V., Langmore, I., Tran, D., Brevdo, E., Vasudevan, S., Moore, D., Patton, B.,
150 Alemi, A., Hoffman, M., & Saurous, R. A. (2017). Tensorflow distributions. *arXiv Preprint*
151 *arXiv:1711.10604*.
- 152 Dirmeier, S. (2024). Surjectors: Surjection layers for density estimation with normalizing flows.
153 *Journal of Open Source Software*, 9(94), 6188.
- 154 Dirmeier, S., Albert, C., & Perez-Cruz, F. (2025). Simulation-based inference for high-
155 dimensional data using surjective sequential neural likelihood estimation. *The 41st*
156 *Conference on Uncertainty in Artificial Intelligence*.
- 157 Dirmeier, S., & Mira, A. (2025). Causal posterior estimation. *arXiv Preprint arXiv:2505.21468*.
- 158 Dutta, R., Schoengens, M., Pacchiardi, L., Ummadisingu, A., Widmer, N., Künzli, P.,
159 Onnela, J.-P., & Mira, A. (2021). ABCpy: A high-performance computing perspective to
160 approximate bayesian computation. *Journal of Statistical Software*, 100(7), 1–38.

- 161 Gloeckler, M., Deistler, M., Weilbach, C. D., Wood, F., & Macke, J. H. (2024). All-in-one
162 simulation-based inference. *Proceedings of the 41st International Conference on Machine*
163 *Learning*.
- 164 Greenberg, D., Nonnenmacher, M., & Macke, J. (2019). Automatic posterior transformation
165 for likelihood-free inference. *Proceedings of the 36th International Conference on Machine*
166 *Learning*.
- 167 Harris, C. R., Millman, K. J., Van Der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau,
168 D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., & others. (2020). Array programming
169 with NumPy. *Nature*, 585(7825), 357–362.
- 170 Hennigan, T., Cai, T., Norman, T., Martens, L., & Babuschkin, I. (2020). *Haiku: Sonnet for*
171 *JAX* (Version 0.0.10). <http://github.com/deepmind/dm-haiku>
- 172 Miller, B. K., Weniger, C., & Forré, P. (2022). Contrastive neural ratio estimation. *Advances*
173 *in Neural Information Processing Systems*.
- 174 Papamakarios, G., Sterratt, D., & Murray, I. (2019). Sequential neural likelihood: Fast
175 likelihood-free inference with autoregressive flows. *Proceedings of the 22nd International*
176 *Conference on Artificial Intelligence and Statistics*.
- 177 Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z.,
178 Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M.,
179 Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). PyTorch: An
180 imperative style, high-performance deep learning library. *Advances in Neural Information*
181 *Processing Systems*.
- 182 Phan, D., Pradhan, N., & Jankowiak, M. (2019). Composable effects for flexible and accelerated
183 probabilistic programming in NumPyro. *arXiv Preprint arXiv:1912.11554*.
- 184 Radev, S. T., Schmitt, M., Schumacher, L., Elsemüller, L., Pratz, V., Schälte, Y., Köthe, U.,
185 & Bürkner, P.-C. (2023). BayesFlow: Amortized Bayesian workflows with neural networks.
186 *Journal of Open Source Software*, 8(89), 5702.
- 187 Schmitt, M., Pratz, V., Köthe, U., Bürkner, P.-C., & Radev, S. T. (2023). Consistency models
188 for scalable and fast simulation-based inference. *arXiv Preprint arXiv:2312.05440*.
- 189 Sharrock, L., Simons, J., Liu, S., & Beaumont, M. (2024). Sequential neural score estimation:
190 Likelihood-free inference with conditional score based diffusion models. *Forty-First*
191 *International Conference on Machine Learning*.
- 192 Sisson, S. A., Fan, Y., & Beaumont, M. (2018). *Handbook of approximate Bayesian*
193 *computation*. CRC Press.
- 194 Sisson, S. A., Fan, Y., & Tanaka, M. M. (2007). Sequential monte carlo without likelihoods.
195 *Proceedings of the National Academy of Sciences*, 104(6), 1760–1765.
- 196 Tejero-Cantero, A., Boelts, J., Deistler, M., Lueckmann, J.-M., Durkan, C., Gonçalves, P. J.,
197 Greenberg, D. S., & Macke, J. H. (2020). Sbi: A toolkit for simulation-based inference.
198 *Journal of Open Source Software*, 5(52), 2505.
- 199 Ulzega, S., Beer, J., Ferriz-Mas, A., Dirmeier, S., & Albert, C. (2025). Shedding light on the
200 solar dynamo using data-driven Bayesian parameter inference. *The Astrophysical Journal*,
201 992(1), 61.
- 202 Wildberger, J. B., Dax, M., Buchholz, S., Green, S. R., Macke, J. H., & Schölkopf, B. (2023).
203 Flow matching for scalable simulation-based inference. *Advances in Neural Information*
204 *Processing Systems*.